

Генеративные модели

Лекция 4: Нормализующие потоки

Проблемы вариационного вывода

Цель:

Хотим аппроксимировать сложное апостериорное распределение $p(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta})$ с помощью более простого $q(\mathbf{z})$

Проблема 1:

Пространство всех возможных функций $q(\mathbf{z})$ бесконечно

Проблема 2:

Для датасета из N объектов $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, нам нужно решать N отдельных задач оптимизации, чтобы найти $\{q_1(\mathbf{z}_1), \dots, q_n(\mathbf{z}_n)\}$

Амортизированный вариационный вывод

Решение 1, параметризация:

Ограничим поиск $q(\mathbf{z})$ по конкретным семействам распределений:

$$q(\mathbf{z}) \rightarrow q_{\phi}(\mathbf{z})$$

Решение 2, амортизация (Amortized Variational Inference):

Вместо отдельных параметров ϕ для каждого $\mathbf{x}$ обучим единую **нейросеть–кодировщик**, которая по $\mathbf{x}$ будет предсказывать параметры для q :

$$q_{\phi}(\mathbf{z}) \rightarrow q_{\phi}(\mathbf{z}|\mathbf{x})$$

Амортизированный вариационный вывод

Введя эти 2 ограничения мы:

1. Ушли от необходимости считать вариационное распределение для каждого объекта $\mathbf{x}$
2. Введя параметры ϕ , мы признаем, что наше решение может быть не точным и KL -дивергенция между $q_\phi(\mathbf{z}|\mathbf{x})$ и $p(\mathbf{z}|\mathbf{x}, \theta)$ может быть больше нуля

Теперь наша задача сводится к нахождению оптимальных ϕ и θ , которые максимизируют $ELBO$

○ Е-шаг:

$$\phi_k = \phi_{k-1} + \eta \nabla_{\phi} \mathcal{L}_{\phi, \theta_{k-1}}(\mathbf{x}) \Big|_{\phi = \phi_{k-1}}$$

○ М-шаг:

$$\theta_k = \theta_{k-1} + \eta \nabla_{\theta} \mathcal{L}_{\phi_k, \theta}(\mathbf{x}) \Big|_{\theta = \theta_{k-1}}$$

Градиент на М-шаге

Считаем градиент по параметрам θ :

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z}, \theta)] - KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

KL -дивергенция не зависит от θ , так как q параметризуется ϕ , а априорное распределение $p(\mathbf{z})$ обычно фиксировано:

$$\nabla_{\theta} \mathcal{L}_{\phi, \theta}(\mathbf{x}) = \nabla_{\theta} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z}, \theta)] = \nabla_{\theta} \int q_{\phi}(\mathbf{z}|\mathbf{x}) \log p(\mathbf{x}|\mathbf{z}, \theta) d\mathbf{z} = \int q_{\phi}(\mathbf{z}|\mathbf{x}) \nabla_{\theta} \log p(\mathbf{x}|\mathbf{z}, \theta) d\mathbf{z}$$

Для оценки матожидания будем использовать метод Монте-Карло:

$$\nabla_{\theta} \mathcal{L}_{\phi, \theta}(\mathbf{x}) \approx \nabla_{\theta} \log p(\mathbf{x}|\mathbf{z}^*, \theta), \quad \mathbf{z}^* \sim q_{\phi}(\mathbf{z}|\mathbf{x})$$

Поскольку $q_{\phi}(\mathbf{z}|\mathbf{x})$ обусловлено на конкретный $\mathbf{x}$, то его вероятностная масса будет сосредоточена в области пространства $\mathbf{z}$, которая наиболее вероятна для этого $\mathbf{x}$, а значит дисперсия будет сильно ниже, чем при наивном подходе

Reparameterization trick

Считаем градиент по параметрам ϕ при фиксированных θ :

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p(\mathbf{x}|\mathbf{z}, \theta)] - KL(q_{\phi}(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))$$

Будем использовать **трюк репараметризации (reparameterization trick)** – считать матожидание по распределению, которое не зависит от ϕ

Предположим, что мы можем выразить $\mathbf{z}$ из распределения $q_{\phi}(\mathbf{z}|\mathbf{x})$ как детерминированную функцию $\mathbf{g}$ от некоторой независимой случайной величины ϵ из распределения $p(\epsilon)$:

$$\mathbf{z} = \mathbf{g}_{\phi}(\mathbf{x}, \epsilon) \quad \epsilon \sim p(\epsilon)$$

Пусть $p(\epsilon) = \mathcal{N}(\mathbf{0}, I)$ и функция репараметризации имеет вид:

$$\mathbf{z} = \mathbf{g}_{\phi}(\mathbf{x}, \epsilon) = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \cdot \epsilon$$

$$q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}(\mathbf{x}), \boldsymbol{\sigma}_{\phi}(\mathbf{x}))$$

Градиент на E-шаге

Считаем градиент по параметрам ϕ при фиксированных θ :

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z}, \theta)] - KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

Используем *reparameterization trick*, чтобы внести оператор градиента внутрь интеграла:

$$\nabla_{\phi} \int q_{\phi}(\mathbf{z}|\mathbf{x}) \log p(\mathbf{x}|\mathbf{z}, \theta) d\mathbf{z} = \int p(\epsilon) \nabla_{\phi} \log p(\mathbf{x}|\mathbf{g}_{\phi}(\mathbf{x}, \epsilon), \theta) d\epsilon$$

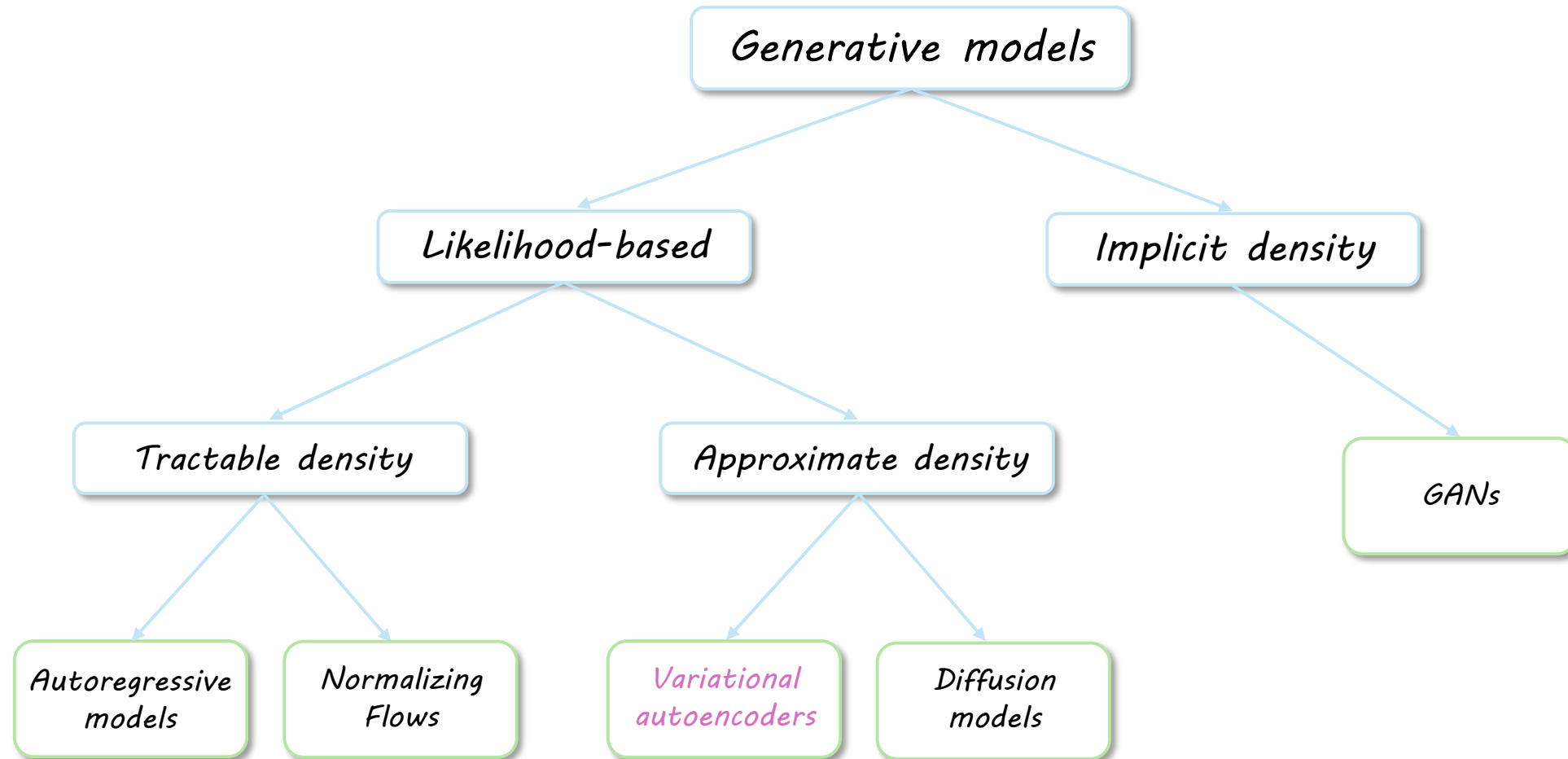
Оценим этот интеграл с помощью метода Монте-Карло:

$$\int p(\epsilon) \nabla_{\phi} \log p(\mathbf{x}|\mathbf{g}_{\phi}(\mathbf{x}, \epsilon), \theta) d\epsilon \approx \nabla_{\phi} \log p(\mathbf{x}|\mu_{\phi}(\mathbf{x}) + \sigma_{\phi}(\mathbf{x}) \cdot \epsilon^*, \theta), \quad \epsilon^* \sim \mathcal{N}(\mathbf{0}, I)$$

Для KL –дивергенции между двумя нормальными распределениями существует аналитическая формула:

$$\nabla_{\phi} KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) = \nabla_{\phi} KL(\mathcal{N}(\mu_{\phi}(\mathbf{x}), \sigma_{\phi}(\mathbf{x}))||\mathcal{N}(\mathbf{0}, I))$$

Зоопарк генеративных моделей

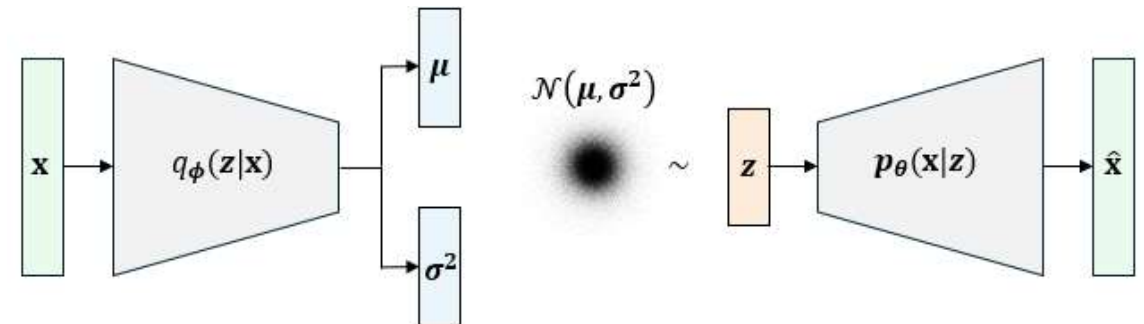


Variational Autoencoder

- *VAE* состоит из двух нейросетей: **кодировщика** и **декодировщика**

Общая идея:

- Одному объекту соответствует не одна точка, а целое распределение $q_{\phi}(\mathbf{z}|\mathbf{x})$
- Кодировщик $q_{\phi}(\mathbf{z}|\mathbf{x})$ выдает не один вектор, а два – вектор средних μ и дисперсий σ^2
- Из этого распределения берется случайная точка $\mathbf{z}$, которая затем подается в декодер для восстановления



Обучение VAE

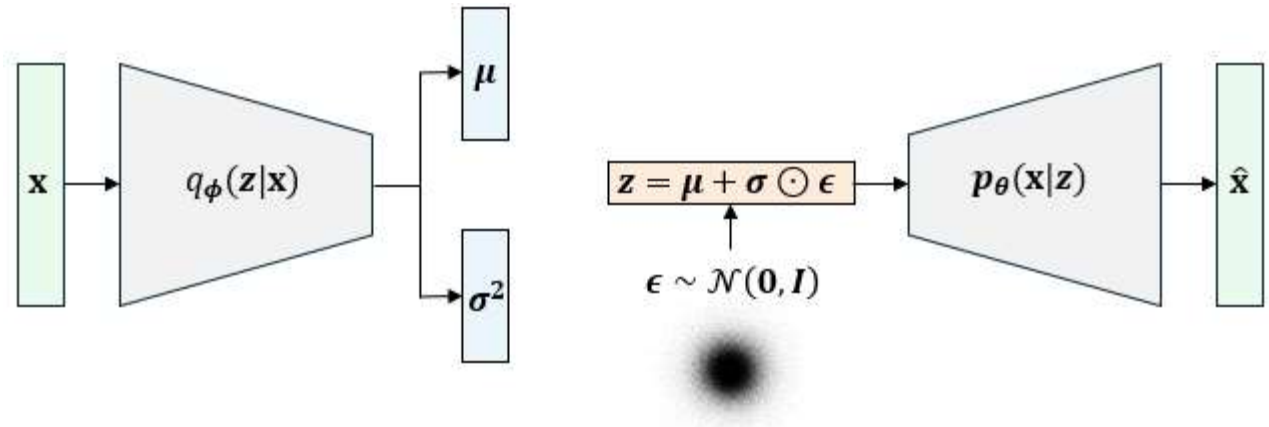
Алгоритм:

- Берем случайный объект $\mathbf{x}$
- Пропускаем его через кодировщик $q_{\phi}(\mathbf{z}|\mathbf{x})$, получаем $\boldsymbol{\mu}_{\phi}(\mathbf{x})$ и $\boldsymbol{\sigma}_{\phi}(\mathbf{x})$
- Вычисляем $\mathbf{z}^*$ с помощью репараметризации:

Берем $\boldsymbol{\epsilon}^* \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

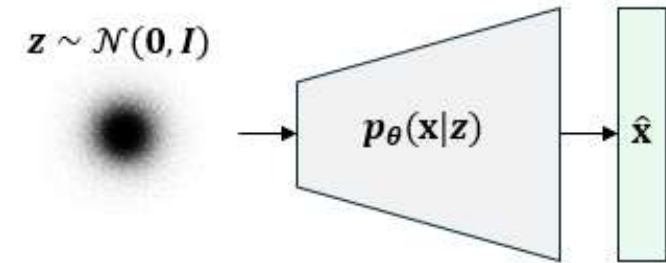
Считаем $\mathbf{z}^* = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \cdot \boldsymbol{\epsilon}^*$

- Пропускаем $\mathbf{z}^*$ через декодер и получаем восстановленный $\hat{\mathbf{x}}$
- Считаем лосс, обновляем параметры ϕ и θ



Генерация VAE

- Кодировщик $q_\phi(\mathbf{z}|\mathbf{x})$ больше не нужен
- Берем случайный вектор $\mathbf{z}^*$ из априорного распределения $\mathcal{N}(\mathbf{0}, \mathbf{I})$
- Подаем $\mathbf{z}^*$ на вход обученному декодеру



План

- Дискретный *VAE*
- Идея нормализующих потоков
- Авторегрессионные потоки
- Линейные потоки
- *NICE*, *RealNVP* и *Glow*

Discrete VAE

Дискретный VAE

- Многие данные по своей природе дискретны
- Трансформеры работают именно с дискретными токенами

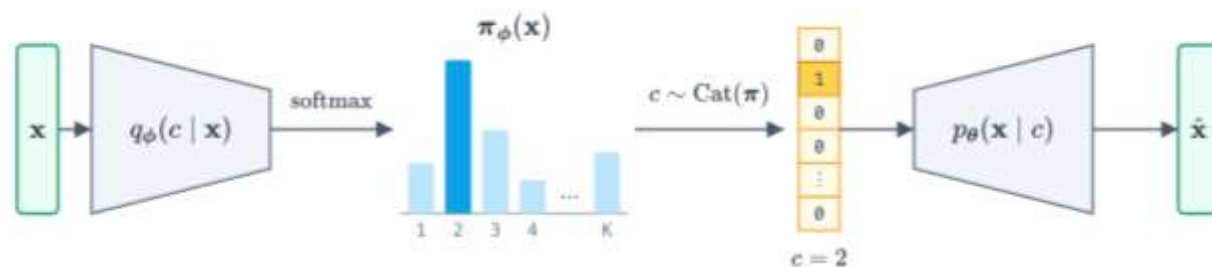
Пусть c — новая скрытая переменная принимает одно из K значений:

$$c \sim \text{Categorical}(\boldsymbol{\pi})$$

$\boldsymbol{\pi}$ — вектор вероятностей:

$$\boldsymbol{\pi} = (\pi_1, \dots, \pi_K), \quad \pi_k = P(c = k),$$

$$\sum_{k=1}^K \pi_k = 1$$



Дискретный VAE

- Кодировщик $q_{\phi}(c|\mathbf{x})$ теперь будет предсказывать вектор вероятностей $\boldsymbol{\pi}$
- Выберем априорное распределение $p(c)$ самым простым – равномерным:

$$p(c) = \frac{1}{K}$$

Преобразуем *ELBO*:

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(c|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|c)] - KL(q_{\phi}(c|\mathbf{x})||p(c))$$

Распишем *KL* –дивергенцию:

$$KL(q_{\phi}(c|\mathbf{x})||p(c)) = \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log \frac{q_{\phi}(k|\mathbf{x})}{p(k)} =$$

$$\sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log q_{\phi}(k|\mathbf{x}) - \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log p(k) = -H(q_{\phi}(c|\mathbf{x})) + \log K$$

Дискретный VAE

Итоговая формула *ELBO* для дискретного VAE:

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(c|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|c)] + H(q_{\phi}(c|\mathbf{x})) - \log K$$

Для одной c матожидание – это сумма по всем классам:

$$\mathbb{E}_{q_{\phi}(c|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|c)] = \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \cdot \log p_{\theta}(\mathbf{x}|k)$$

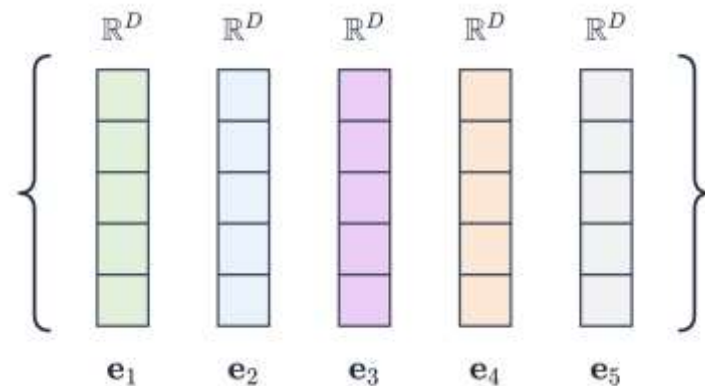
- У изображения скрытых переменных $\mathbf{c} = (c_1, \dots, c_{H \times W})$ и слагаемых в сумме становится $K^{H \times W}$
- Нужно делать оценку $\mathbb{E}_{q_{\phi}(c|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{c})] \approx \log p_{\theta}(\mathbf{x}|\mathbf{c}^*)$, $\mathbf{c}^* \sim q_{\phi}(\mathbf{c}|\mathbf{x})$
- Сэмплирование недифференцируемо и трюк репараметризации больше не работает

Векторное квантование

Идея **векторной квантизации** (*Vector Quantization, VQ*):

- Сделаем выбор класса детерминированным
- Будем аппроксимировать непрерывное пространство конечным набором векторов

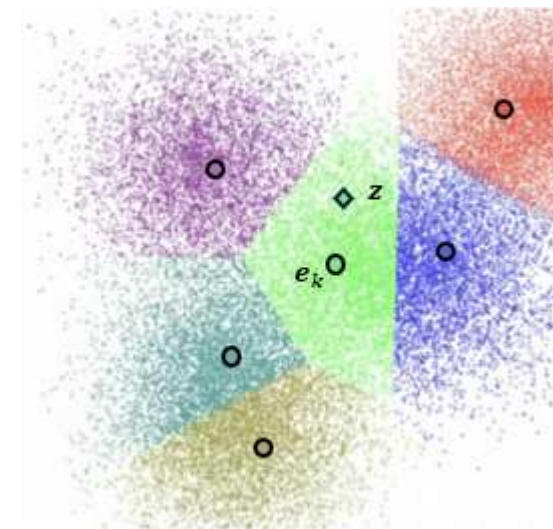
- Создадим кодовую книгу (**codebook**) – словарь из K обучаемых векторов $\{\mathbf{e}_k\}_{k=1}^K$
- Каждый $\mathbf{e}_k$ – вектор в пространстве $\mathbb{R}^D$:
 K – размер словаря
 D – размерность каждого вектора



Векторное квантование

Для вектора $\mathbf{z}_e \in \mathbb{R}^D$ находим ближайший к нему вектор из кодовой книги и получаем квантованный вектор $\mathbf{z}_q$:

$$\mathbf{z}_q = q(\mathbf{z}_e) = \mathbf{e}_{k^*}, \quad k^* = \arg \min_k \|\mathbf{z}_e - \mathbf{e}_k\|$$

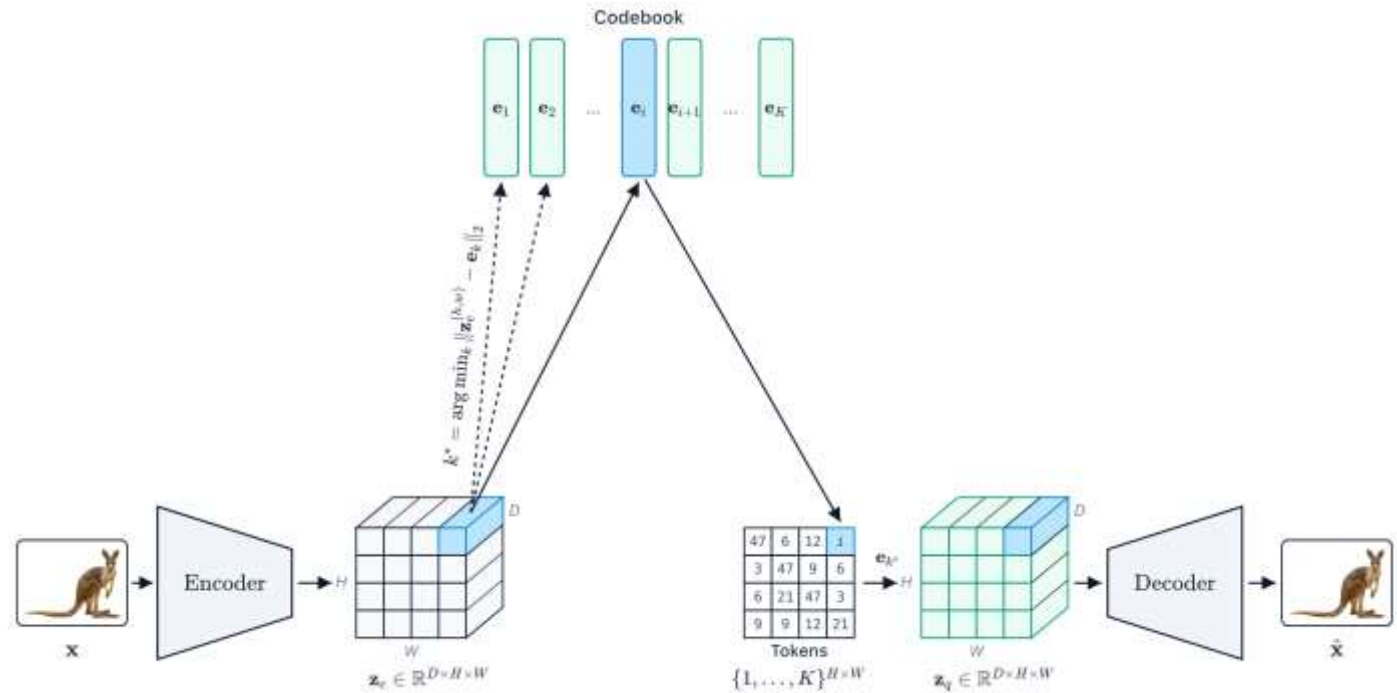


Это детерминированная операция, похожая на поиск ближайшего соседа в ***K – Means***

k^* – это и есть дискретная переменная s , только вместо сэмплирования выбираем ближайший вектор

Векторное квантование

- **Encoder**: принимает на вход объект $\mathbf{x}$, сжимает его и выдает непрерывный вектор $\mathbf{z}_e \in \mathbb{R}^{D \times H \times W}$
- **VQ**: для каждого из $H \times W$ векторов находит ближайший к нему вектор $\mathbf{e}_k^*$ из кодовой книги и сохраняет его номер в сетку токенов
- **Decoder**: на вход декодеру приходит уже квантованный вектор $\mathbf{z}_q = \mathbf{e}_k^*$, из которого восстанавливается исходный объект



Векторное квантование

Особенностью $VQ - VAE$ является использование детерминированного вариационного апостериорного распределения:

$$q_{\phi}(c = k|\mathbf{x}) = \begin{cases} 1 & \text{для } k = k^* = \underset{k}{\operatorname{arg\,min}} ||\mathbf{z}_e - \mathbf{e}_k|| \\ 0 & \text{иначе} \end{cases}$$

Такой выбор значительно упрощает $ELBO$:

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(c|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|c)] + \underbrace{H(q_{\phi}(c|\mathbf{x}))}_0 - \log K = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - \log K$$

Итог:

- Вместе с сэмплированием исчезла стохастичность кодировщика
- Регуляризатором является кодовая книга

Straight – Through Estimator

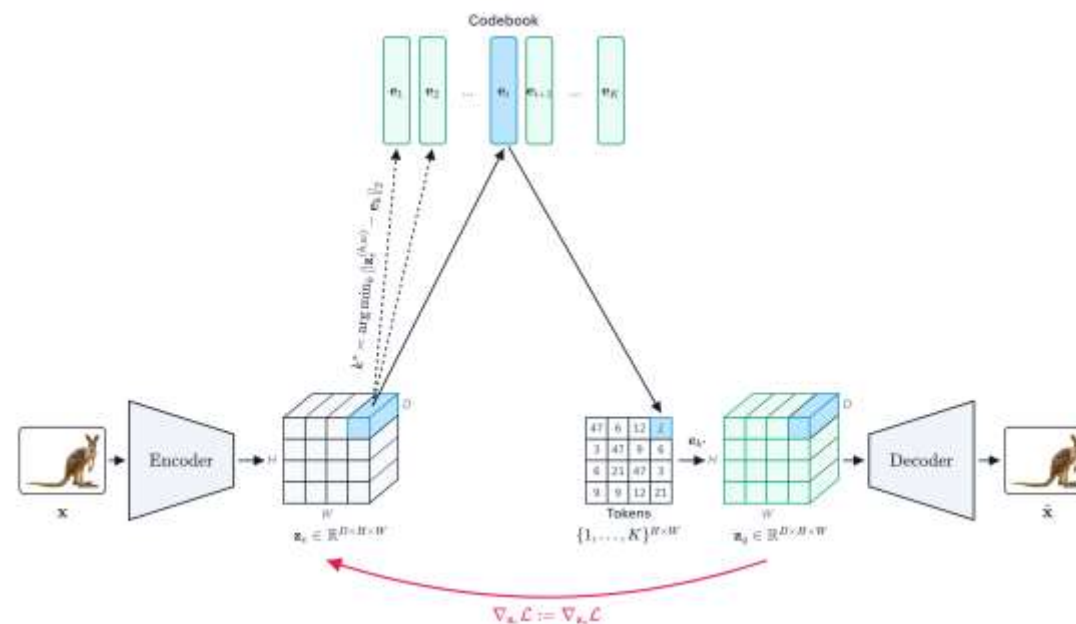
Операция квантования $k^* = \arg \min_k ||\mathbf{z}_e - \mathbf{e}_k||$ является недифференцируемой

Авторы $VQ - VAE$ применили *прямое оценивание градиента (Straight – Through Estimator, STE)*

Механизм **STE**:

- На этапе обратного распространения градиент просто копируется с выхода недифференцируемого блока на его вход
- В $VQ - VAE$ градиент по $\mathbf{z}_q$ напрямую передается $\mathbf{z}_e$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_e} \approx \frac{\partial \mathcal{L}}{\partial \mathbf{z}_q}$$



Straight – Through Estimator

Математически это можно представить как аппроксимацию в *chain rule*:

$$\frac{\partial \log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}{\partial \phi} = \frac{\partial \log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}{\partial \mathbf{z}_q} \cdot \frac{\partial \mathbf{z}_q}{\partial \mathbf{z}_e} \cdot \frac{\partial \mathbf{z}_e}{\partial \phi} \approx \frac{\partial \log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}{\partial \mathbf{z}_q} \cdot \frac{\partial \mathbf{z}_e}{\partial \phi}$$

- Выход кодировщика $\mathbf{z}_e$ и квантованный вектор $\mathbf{z}_q$ находятся рядом в одном и том же пространстве $\mathbb{R}^D$
- Градиент от декодера $\nabla_{\mathbf{z}_q} \mathcal{L}$ показывает, куда сдвинуть $\mathbf{z}_q$, чтобы улучшить реконструкцию
- Кодировщик применяет это к $\mathbf{z}_e$ и подталкивает модель выбирать более подходящий вектор $\mathbf{e}_k$

Хотя такая оценка является смещенной, она часто указывает в направлении истинного градиента и хорошо работает на практике

Straight – Through Estimator

Цепочка вычислений:

$$\mathbf{z}_e = \text{Enc}_\phi(\mathbf{x}) \rightarrow \mathbf{z}_q = \mathbf{e}_{k^*} \rightarrow \mathcal{L}_{rec} = -\log p_\theta(\mathbf{x}|\mathbf{z}_q)$$

Проход градиента от $\mathcal{L}_{rec}$:

- Декодер получает напрямую через $\nabla_\theta \mathcal{L}_{rec} = -\nabla_\theta \log p_\theta(\mathbf{x}|\mathbf{z}_q)$
- Кодировщик получает через STE $\nabla_\phi \mathcal{L}_{rec} \approx \frac{\partial \mathcal{L}_{rec}}{\partial \mathbf{z}_q} \cdot \frac{\partial \mathbf{z}_e}{\partial \phi}$
- В кодовой книге градиент проходит мимо блока VQ

Для обучения кодовой книги нужен отдельный член в функции потерь

VQ – VAE

Нам нужно обучать кодировщик ϕ , декодер θ и кодовую книгу $\{\mathbf{e}_k\}_{k=1}^K$

В VQ – VAE используется составная функция потерь:

$$\mathcal{L} = \underbrace{-\log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}_{\text{Reconstruction loss}} + \underbrace{\|sg[\mathbf{z}_e] - \mathbf{e}_{k^*}\|^2}_{\text{VQ – Loss}} + \underbrace{\beta \|\mathbf{z}_e - sg[\mathbf{e}_{k^*}]\|^2}_{\text{Commitment Loss}}$$

sg – оператор **stop – gradient**

VQ – Loss:

- Обновляет только *codebook*
- Заставляет выбранный вектор $\mathbf{e}_{k^*}$ сдвинуться ближе к $\mathbf{z}_e$

Commitment – Loss:

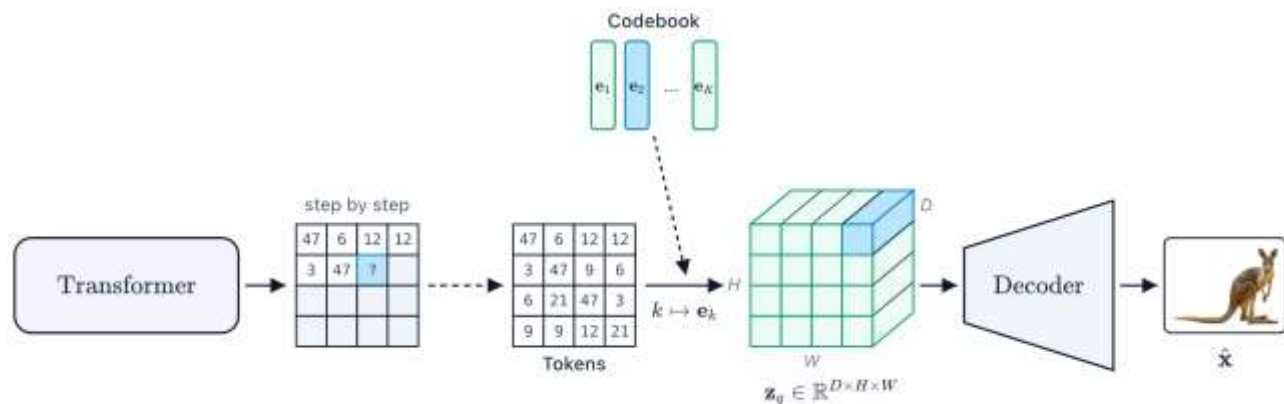
- Обновляет только кодировщик
- Заставляет выход кодировщика $\mathbf{z}_e$ сдвинуться ближе к $\mathbf{e}_{k^*}$

Генерация VQ – VAE

- Сэмплировать токены из $p(c) = \frac{1}{K}$ – плохая идея, так как токены выбираются независимо
- В реальных изображениях соседние токены согласованы

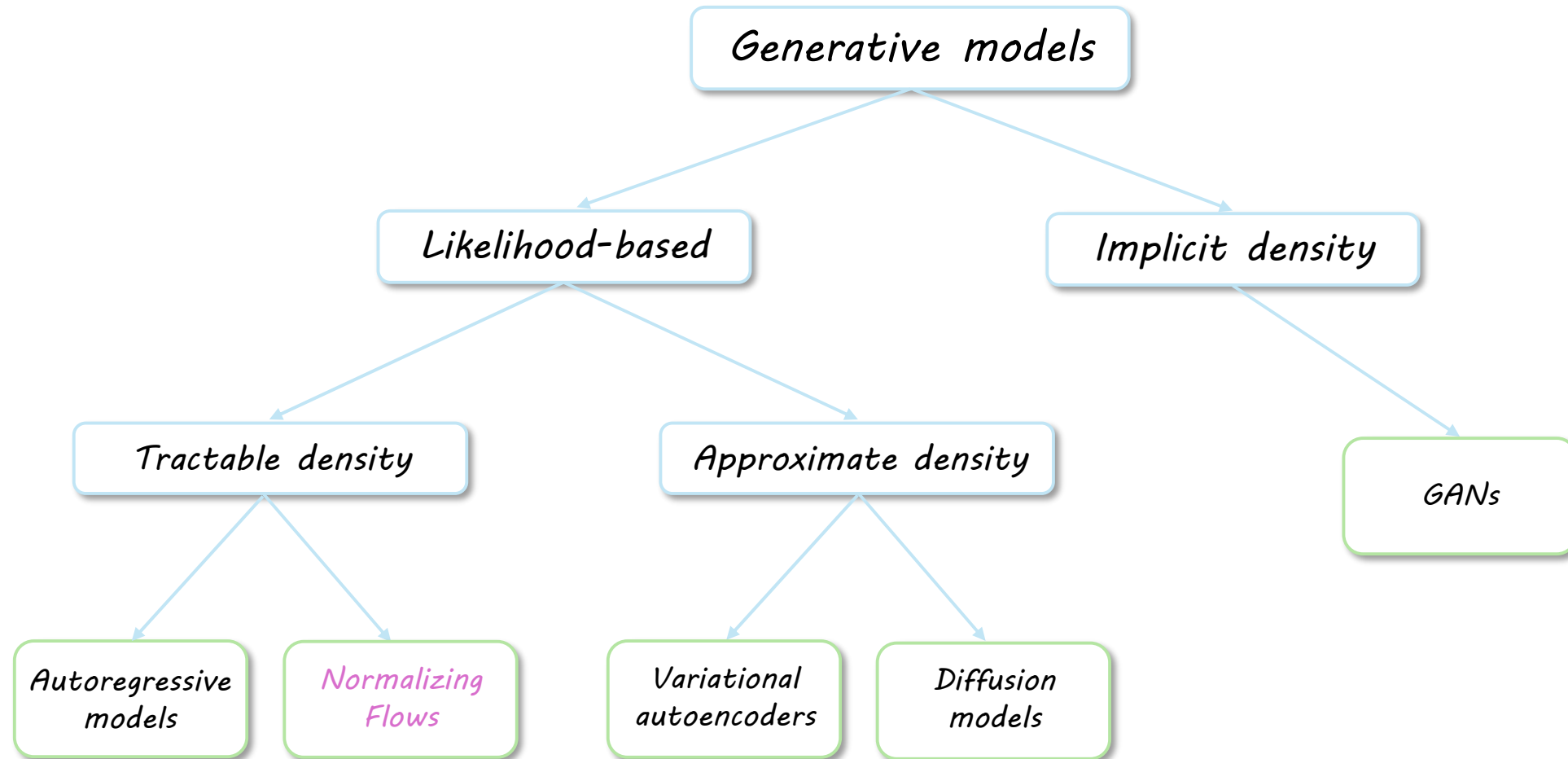
Решение:

- Выучить распределение токенов отдельной моделью
- Обучаем VQ-VAE и кодируем всю выборку в сетки токенов $H \times W$
- На этих сетках обучаем AR модель



Нормализующие потоки

Зоопарк генеративных моделей



Разложение многомерной плотности

Авторегрессионные модели:

- ⊕ Точно вычисляют правдоподобие
- ⊖ Медленная генерация

Вариационные автокодировщики:

- ⊕ Быстрая генерация из латентного пространства
- ⊖ Не могут точно вычислить правдоподобие

Нормализующие потоки:

- ⊕ Точно вычисляют правдоподобие
- ⊕ Используют латентное пространство для быстрой генерации

Нормализующие потоки берут лучшее от двух миров

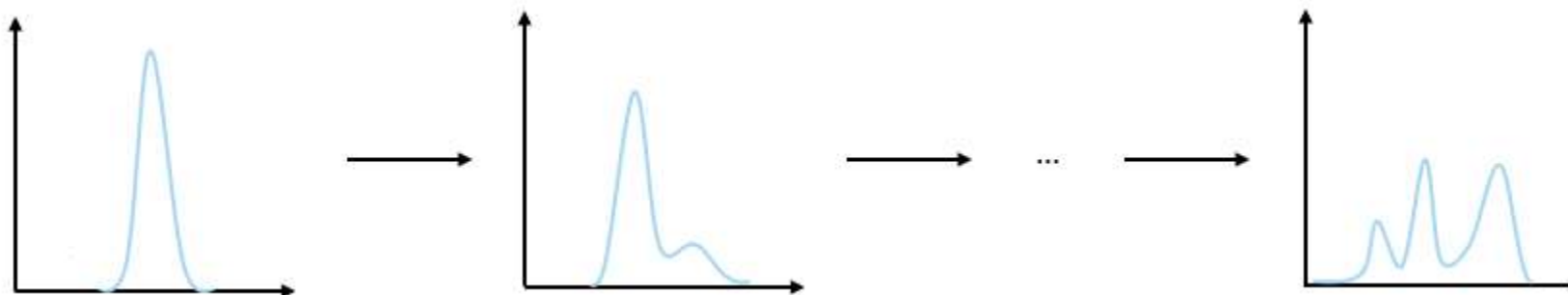
Идея нормализующих потоков

Идея:

Возьмём простое, хорошо изученное распределение:

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Шаг за шагом будем усложнять это распределение, пока оно не станет похоже на наши данные $\mathbf{x}$



Идея нормализующих потоков

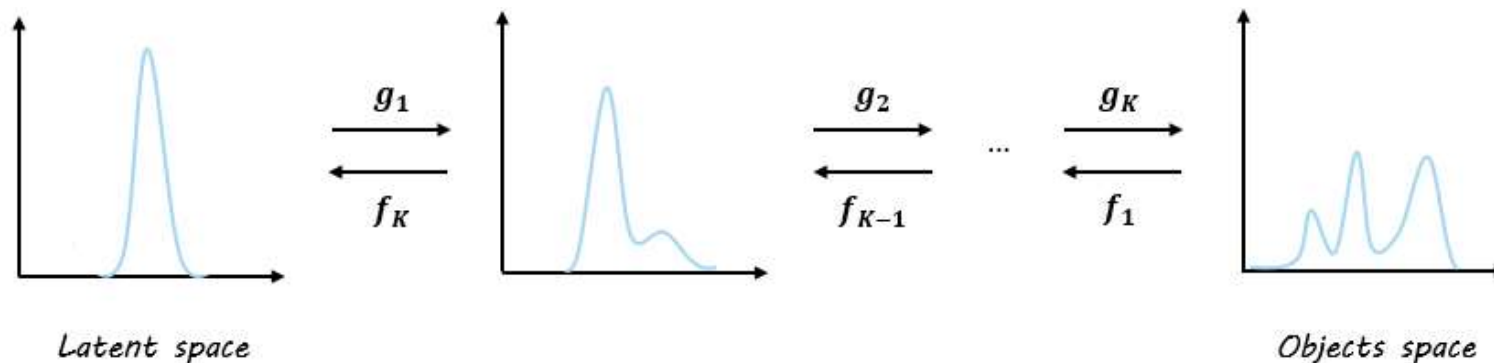
Особенность потоков – наличие двух направлений:

- **Нормализующее направление (для обучения)** $x \rightarrow z$:

Берём реальный объект x и с помощью нормализующей функции f превращаем его в простой z

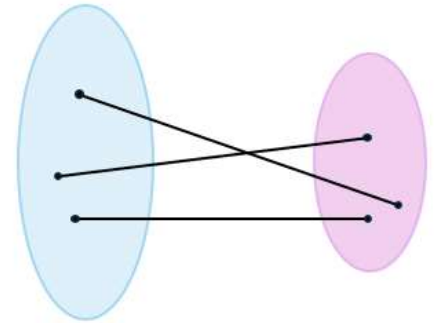
- **Генеративное направление** $z \rightarrow x$:

Берём простой объект z и с помощью генеративной функции $g = f^{-1}$ создаём объект x



Идея нормализующих потоков

Функции взаимно обратные $\rightarrow$ для каждой точки в пространстве $\mathbf{x}$ существует ровно одна из пространства $\mathbf{z}$



Размерности исходного и преобразованного пространств должны совпадать:

$$\dim \mathbf{x} = \dim \mathbf{z}$$

Теорема о замене переменных

Наша цель – найти распределение $p(\mathbf{x})$

Связать неизвестную плотность $p(\mathbf{x})$ с известной плотностью простого распределения $p(\mathbf{z})$ нам позволяет **формула замены переменных**:

$$p(\mathbf{x}) = p(\mathbf{z}) |\det(J_f)| = p(\mathbf{z}) \left| \det \left(\frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right) \right| = p(\mathbf{f}(\mathbf{x})) \left| \det \left(\frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

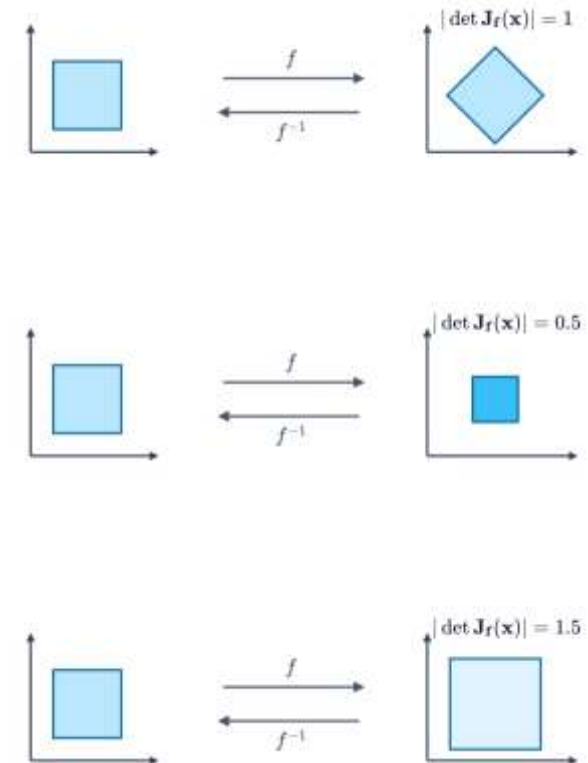
Здесь $J_f(\mathbf{x})$ – матрица Якоби:

$$J_f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_D} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_D}{\partial x_1} & \dots & \frac{\partial f_D}{\partial x_D} \end{pmatrix}$$

Якобиан

- Интуитивно определитель Якобиана можно понимать как коэффициент изменения объема
- Он показывает, насколько f сжимает или растягивает пространство

- Если объем не изменился $|\det J_f(x)| = 1$, плотность в этой точке не изменится
- Если пространство сжалось $|\det J_f(x)| < 1$, плотность в этой точке должна увеличиться
- Если пространство растянулось $|\det J_f(x)| > 1$, плотность в этой точке должна уменьшиться



Пример

Пусть есть одномерная случайная величина $z \sim \mathcal{N}(0, 1)$:

$$\pi(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}}$$

Хотим получить новую переменную:

$$x = g(z) = 5z + 1$$

Найдем нормализующую функцию f , которая является обратной к g :

$$z = f(x) = g^{-1}(x) = \frac{x - 1}{5}$$

Пример

Якобиан в одномерном случае:

$$\left| \frac{df(x)}{dx} \right| = \left| \frac{d}{dx} \left(\frac{x-1}{5} \right) \right| = \left| \frac{1}{5} \right| = \frac{1}{5}$$

Подставляем и получаем распределение для x :

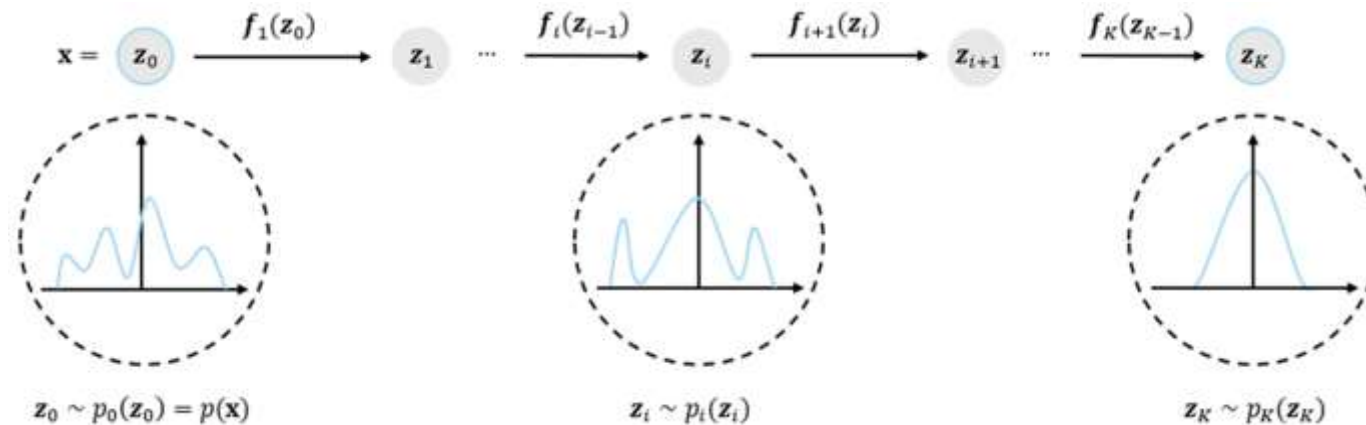
$$p(x) = \frac{1}{5} \pi \left(\frac{x-1}{5} \right) = \frac{1}{\sqrt{2\pi \cdot 5^2}} e^{-\frac{(x-1)^2}{2 \cdot 5^2}} = \mathcal{N}(1, 25)$$

Композиция преобразований

- Одной функции f недостаточно, чтобы из $\mathcal{N}(0,1)$ получить сложное распределение наших данных
- Потoki используют последовательность из K простых и обратимых преобразований:

$$\mathbf{x} = \mathbf{z}_0 \xrightarrow{f_1} \mathbf{z}_1 \xrightarrow{f_2} \dots \xrightarrow{f_K} \mathbf{z}_K$$

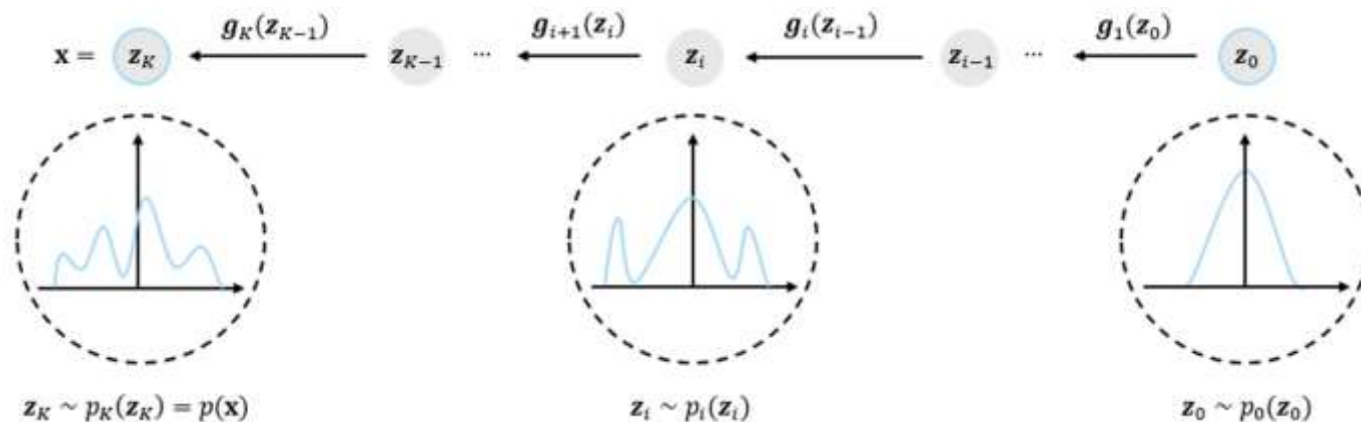
- Во время обучения мы используем нормализующее направление $\mathbf{x} \rightarrow \mathbf{z}$:



Композиция преобразований

- Генеративное направление $\mathbf{z} \rightarrow \mathbf{x}$ используется для создания новых объектов:

$$\mathbf{z}_0 \xrightarrow{g_1} \mathbf{z}_1 \xrightarrow{g_2} \dots \xrightarrow{g_K} \mathbf{z}_K = \mathbf{x}$$



Композиция преобразований

Нужно найти определитель якобиана всей композиции

Два математических факта:

- Матрица Якоби для композиций функций равна их произведению:

$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \frac{\partial \mathbf{f}_K}{\partial \mathbf{z}_{K-1}} \cdot \frac{\partial \mathbf{f}_{K-1}}{\partial \mathbf{z}_{K-2}} \cdot \dots \cdot \frac{\partial \mathbf{f}_1}{\partial \mathbf{z}_0}$$

- Определитель произведения матриц равен произведению определителей:

$$\det(\mathbf{J}_f) = \det(\mathbf{J}_{f_K}) \cdot \det(\mathbf{J}_{f_{K-1}}) \cdot \dots \cdot \det(\mathbf{J}_{f_1})$$

Будем считать определители для каждого шага $\mathbf{f}_k$ и перемножать их:

$$\det\left(\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right) = \prod_{k=1}^K \det\left(\frac{\partial \mathbf{f}_k}{\partial \mathbf{z}_{k-1}}\right)$$

Обучение нормализующих потоков

Чтобы сделать эту композицию обучаемой, представим f_k как нейросеть с параметрами θ

Плотность, которую моделирует наша модель:

$$p_{\theta}(\mathbf{x}) = p(\mathbf{f}_{\theta}(\mathbf{x})) \left| \det \left(\frac{\partial \mathbf{f}_{\theta}}{\partial \mathbf{x}} \right) \right| = p(\mathbf{z}_K) \left| \prod_{k=1}^K \det \left(\frac{\partial \mathbf{f}_{k,\theta}}{\partial \mathbf{z}_{k-1}} \right) \right|$$

$\mathbf{z}_K$ — финальный вектор в латентном пространстве

Максимизируем правдоподобие:

$$\log p_{\theta}(\mathbf{x}) = \log p(\mathbf{z}_K) + \sum_{k=1}^K \log \left| \det \left(\frac{\partial \mathbf{f}_{k,\theta}}{\partial \mathbf{z}_{k-1}} \right) \right|$$

- $\log p(\mathbf{z}_K)$ отвечает за то, чтобы выход потока соответствовал базовому распределению
- $\sum_{k=1}^K \log \left| \det \left(\frac{\partial \mathbf{f}_{k,\theta}}{\partial \mathbf{z}_{k-1}} \right) \right|$ отслеживает суммарное изменение объема после всех преобразований

Функция потерь

Функция потерь:

$$\mathcal{L}_\theta(\mathbf{x}) = -\log p_\theta(\mathbf{x}) = -\left(\log p(\mathbf{z}_K) + \sum_{k=1}^K \log \left| \det \left(\frac{\partial \mathbf{f}_{k,\theta}}{\partial \mathbf{z}_{k-1}} \right) \right| \right)$$

Требования к преобразованиям $\mathbf{f}_{k,\theta}$:

- Наличие обратной функции $\mathbf{g}_\theta = \mathbf{f}_\theta^{-1}$
- Быстрое вычисление определителя $\det(\mathbf{J}_f)$ (в идеале за $\mathcal{O}(D)$)



Будем искать функции, которые удовлетворяют этим условиям

Авторегрессионные потоки

Авторегрессионные потоки

Авторегрессионные потоки вдохновлены идеей классических AR моделей:

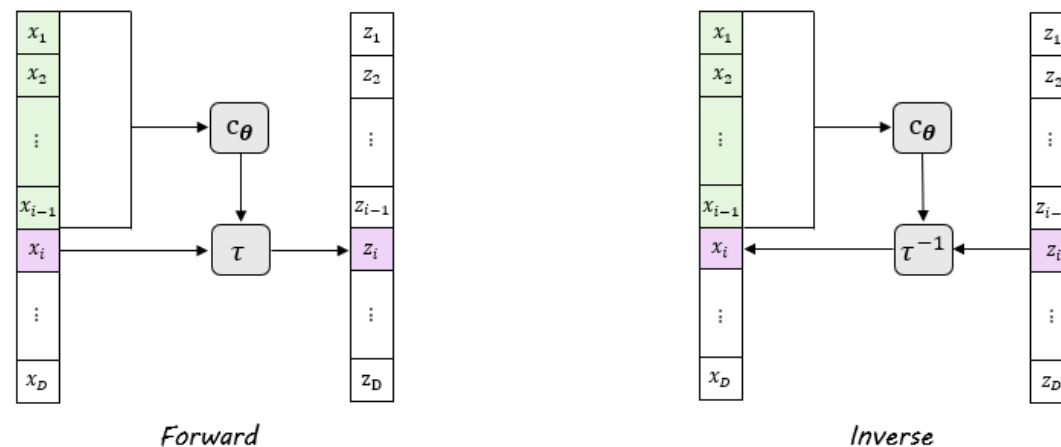
$$z_i = \tau(x_i; c_\theta(\mathbf{x}_{1:i-1}))$$

τ – **transformer**, обратимая функция, которая преобразует x_i в z_i

$c_\theta(\mathbf{x}_{1:i-1})$ – **conditioner**, модель, которая генерирует параметры для τ на основе $\mathbf{x}_{1:i-1}$

Если τ обратим, то и весь поток обратим:

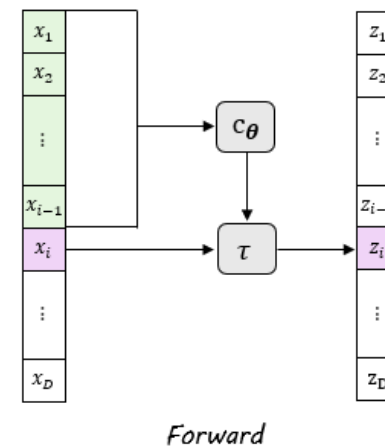
$$x_i = \tau^{-1}(z_i; c_\theta(\mathbf{x}_{1:i-1}))$$



Авторегрессионные потоки

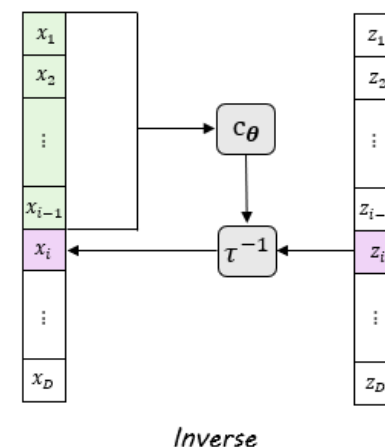
- При прямом проходе $\mathbf{x}$ полностью известен и мы можем параллельно вычислить все z_i

$$z_i = \tau(x_i; c_{\theta}(\mathbf{x}_{1:i-1}))$$



- При обратном проходе $\mathbf{z}$ полностью известен, но для генерации x_i нам нужен контекст $\mathbf{x}_{1:i-1}$

$$x_i = \tau^{-1}(z_i; c_{\theta}(\mathbf{x}_{1:i-1}))$$



Авторегрессионные потоки

Якобиан имеет треугольный вид, поскольку $\frac{\partial z_i}{\partial x_j} = 0$ при $j > i$:

$$J_{f_\theta}(\mathbf{x}) = \begin{pmatrix} \frac{\partial z_1}{\partial x_1} & \dots & 0 \\ \vdots & \ddots & \vdots \\ \frac{\partial z_D}{\partial x_1} & \dots & \frac{\partial z_D}{\partial x_D} \end{pmatrix}$$

Определитель треугольной матрицы считается за $\mathcal{O}(D)$:

$$\log |\det J_{f_\theta}(\mathbf{x})| = \sum_{i=1}^D \log \left| \frac{\partial \tau(x_i; c_\theta(\mathbf{x}_{1:i-1}))}{\partial x_i} \right|$$

Affine Transformers

Аффинное преобразование:

$$z_i = \alpha_i \cdot x_i + \beta_i$$

α_i и β_i — параметры масштаба и сдвига

Такой поток всегда обратим (если $\alpha_i \neq 0$):

$$x_i = \frac{z_i - \beta_i}{\alpha_i}$$

Частная производная $\frac{\partial z_i}{\partial x_i} = \alpha_i$

Логарифм определителя якобиана для всего преобразования:

$$\log |\det J_{f_\theta}(\mathbf{x})| = \sum_i^D \log |\alpha_i|$$

Masked Conditioners

Хотим посчитать параметры $c_{\theta}(\mathbf{x}_{1:i-1})$ для генерации z_i :

$$z_i = \tau(x_i; c_{\theta}(\mathbf{x}_{1:i-1}))$$

Будем использовать единую нейросеть для вычисления всех параметров

Будем использовать маски, чтобы сохранить авторегрессию в этой сети:

- В полносвязных сетях на веса накладываются бинарные маски (**MADE**)
- В свёрточных сетях используются **causal convolution**

Либо на обучении, либо на генерации будет медленная скорость

Есть 2 популярные архитектуры:

- **MAF (Masked Autoregressive Flow)** — создана для быстрого обучения
- **IAF (Inverse Autoregressive Flow)** — та же архитектура, но развернута для быстрой генерации

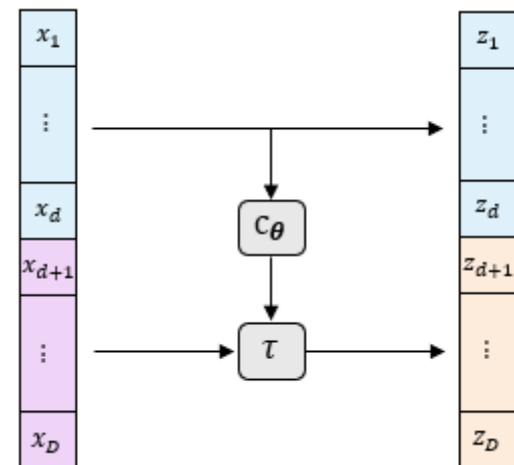
Coupling Layers

Чтобы решить проблему вычислительной асимметрии были предложены **слои связи** (*coupling layers*)

Делим входной вектор $\mathbf{x}$ на две части:

$$\mathbf{x} = [\mathbf{x}_{1:d}; \mathbf{x}_{d+1:D}]$$

- Первая часть $\mathbf{x}_{1:d}$ преобразуется напрямую без использования c_θ
- Вторая часть $\mathbf{x}_{d+1:D}$ преобразуется с помощью τ



Обычно преобразование первой части тождественное:

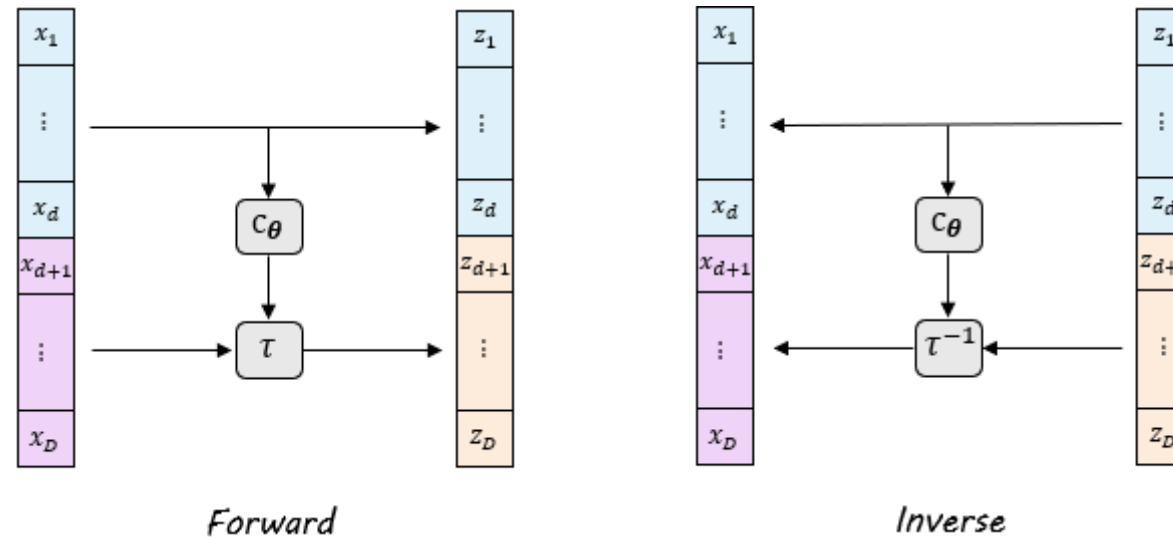
$$\begin{cases} \mathbf{z}_{1:d} = \mathbf{x}_{1:d} \\ \mathbf{z}_{d+1:D} = \tau(\mathbf{x}_{d+1:D}; c_\theta(\mathbf{x}_{1:d})) \end{cases}$$

Свойства слоёв связи

Симметричные вычисления:

- Прямой проход быстр, так как $\mathbf{x}$ известен
- Обратный проход тоже быстр, так как мы знаем первую половину $\mathbf{x}_{1:d}$:

$$\begin{cases} \mathbf{x}_{1:d} = \mathbf{z}_{1:d} \\ \mathbf{x}_{d+1:D} = \tau^{-1}(\mathbf{z}_{d+1:D}; c_{\theta}(\mathbf{x}_{1:d})) = \tau^{-1}(\mathbf{z}_{d+1:D}; c_{\theta}(\mathbf{z}_{1:d})) \end{cases}$$



Свойства слоёв связи

Блочный нижнетреугольный якобиан:

$$J_f = \begin{pmatrix} \frac{\partial \mathbf{z}_{1:d}}{\partial \mathbf{x}_{1:d}} & \frac{\partial \mathbf{z}_{1:d}}{\partial \mathbf{x}_{d+1:D}} \\ \frac{\partial \mathbf{z}_{d+1:D}}{\partial \mathbf{x}_{1:d}} & \frac{\partial \mathbf{z}_{d+1:D}}{\partial \mathbf{x}_{d+1:D}} \end{pmatrix} = \begin{pmatrix} I & \mathbf{0} \\ A & \Lambda \end{pmatrix}$$

Определитель равен произведению определителей диагональных блоков:

$$\det(J_f) = \det(I) \cdot \det(\Lambda) = \det(\Lambda)$$

Линейные потоки

Линейные потоки

- Авторегрессионные потоки и слои связи чувствительны к порядку переменных
- Фиксированная перестановка между слоями – необучаемая операция

Решение:

- Вставить между нелинейными слоями обучаемый линейный поток:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$\mathbf{W}$ – обратимая матрица, $\mathbf{b}$ – вектор смещения

- Это позволит модели самой находить оптимальное перемешивание

Линейные потоки

- Якобиан равен самой матрице:

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \mathbf{W} \quad \Rightarrow \quad \log p(\mathbf{x}) = \log p(\mathbf{z}) + \log |\det \mathbf{W}|$$

- Перестановка только меняет компоненты местами:

$$\begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} x_3 \\ x_2 \\ x_1 \end{pmatrix}$$

- Обучаемая матрица $\mathbf{W}$ смешивает компоненты:

$$z_1 = w_{11} \cdot x_1 + w_{12} \cdot x_2 + w_{13} \cdot x_3$$

- Линейные потоки ограничены в выразительности (гауссово распределение остаётся)
- Используются в основном как обучаемое перемешивание между слоями

Линейные потоки

Проблема:

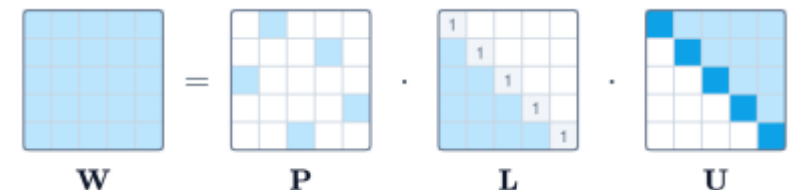
- Определитель якобиана считается за $\mathcal{O}(D^3)$

Решение:

- Не будем обучать матрицу напрямую, параметризуем её через матричные разложения

Самый популярный способ – использовать **PLU-разложение**:

$$\mathbf{W} = \mathbf{PLU} \Rightarrow \log|\det \mathbf{W}| = \sum_{i=1}^D \log |U_{ii}|$$



$\mathbf{P}$ – фиксированная матрица перестановок (не обучается)

$\mathbf{L}$ – нижнетреугольная матрица с единицами на диагонали

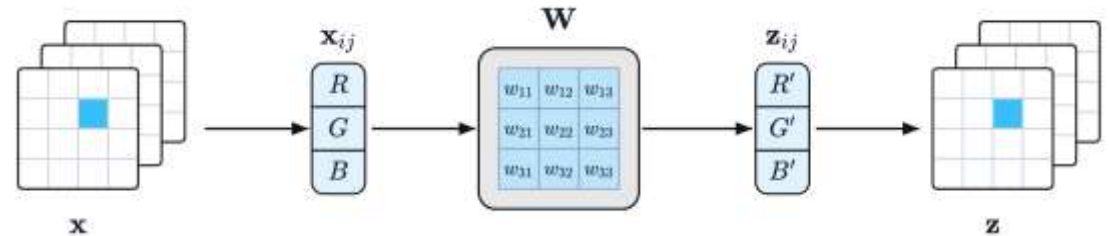
$\mathbf{U}$ – верхнетреугольная матрица

Линейные потоки

- Матрица P не обучается
- Позднее были работы, которые использовали ***QR-разложение***:

$$W = QR$$

- Для изображений обычно применяется ***обратимая 1×1 свертка***
- Это линейный поток, который перемешивает информацию между каналами
- Одна и та же матрица W применяется к каждому пикселю (i, j)



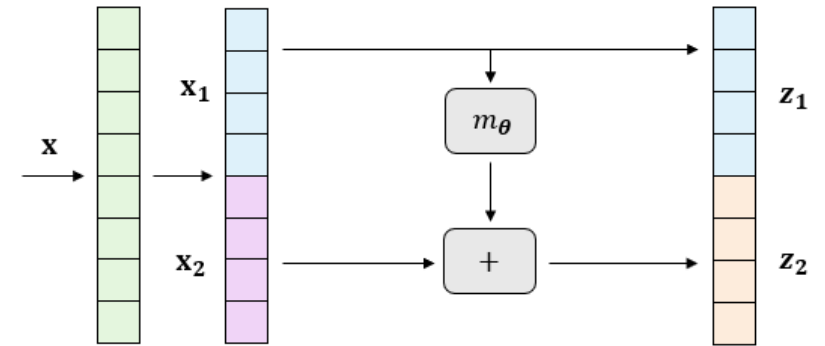
NICE и RealNVP

NICE: Additive Coupling Layers

Разделяем вектор $\mathbf{x}$ на 2 части $\mathbf{x}_{1:d}$ и $\mathbf{x}_{d+1:D}$:

$$\begin{cases} \mathbf{z}_{1:d} = \mathbf{x}_{1:d} \\ \mathbf{z}_{d+1:D} = \mathbf{x}_{d+1:D} + m_{\theta}(\mathbf{x}_{1:d}) \end{cases}$$

m_{θ} — произвольная нейросеть



Такая архитектура легко обратима:

$$\begin{cases} \mathbf{x}_{1:d} = \mathbf{z}_{1:d} \\ \mathbf{x}_{d+1:D} = \mathbf{z}_{d+1:D} - m_{\theta}(\mathbf{z}_{1:d}) \end{cases}$$

Якобиан имеет блочный нижнетреугольный вид:

$$J = \begin{pmatrix} \mathbf{I}_d & \mathbf{0} \\ \frac{\partial \mathbf{z}_{d+1:D}}{\partial \mathbf{x}_{1:d}} & \mathbf{I}_{D-d} \end{pmatrix}$$

NICE: Проблемы

Определитель якобиана всегда равен единице, поэтому такое преобразование сохраняет объем

Даже композиция из нескольких аддитивных слоёв все равно сохраняет объем

Авторы *NICE* предложили в конце добавить слой масштабирования:

$$\mathbf{z} = \mathbf{s} \odot \mathbf{h}$$

$\mathbf{h}$ — выход последнего связующего слоя

$\mathbf{s}$ — обучаемый вектор масштаба

$\odot$ — поэлементное умножение

Определитель якобиана:

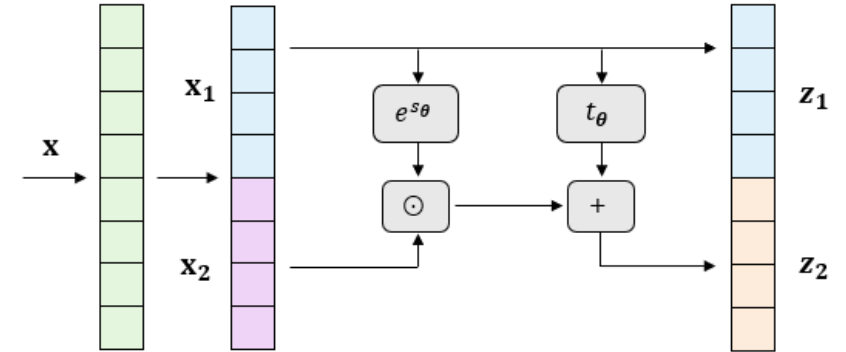
$$\det(\mathbf{J}) = \prod_{i=1}^D s_i$$

RealNVP: Affine Coupling Layers

Разделяем вектор $\mathbf{x}$ на 2 части $\mathbf{x}_{1:d}$ и $\mathbf{x}_{d+1:D}$:

$$\begin{cases} \mathbf{z}_{1:d} = \mathbf{x}_{1:d} \\ \mathbf{z}_{d+1:D} = \mathbf{x}_{d+1:D} \odot e^{s_\theta(\mathbf{x}_{1:d})} + t_\theta(\mathbf{x}_{1:d}) \end{cases}$$

s_θ , t_θ — произвольные нейросети



Такая архитектура всё равно легко обратима:

$$\begin{cases} \mathbf{x}_{1:d} = \mathbf{z}_{1:d} \\ \mathbf{x}_{d+1:D} = (\mathbf{z}_{d+1:D} - t_\theta(\mathbf{z}_{1:d})) \odot e^{-s_\theta(\mathbf{z}_{1:d})} \end{cases}$$

И якобиан имеет блочный нижнетреугольный вид:

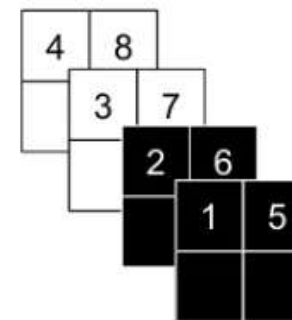
$$J = \begin{pmatrix} \mathbf{I}_d & \mathbf{0} \\ \frac{\partial \mathbf{z}_{d+1:D}}{\partial \mathbf{x}_{1:d}} & \text{diag}(e^{s_\theta(\mathbf{x}_{1:d})}) \end{pmatrix}$$

RealNVP

- Для эффективной работы с изображениями в **RealNVP** используют иерархическую (**Multi – Scale**) архитектуру
- Это позволяет переходить от анализа мелких деталей к более крупным

Типичный блок устроен следующим образом:

- Сначала применяются несколько слоёв связи с **шахматными масками**
- Затем сжимаем $h \times w \times c \rightarrow \frac{h}{2} \times \frac{w}{2} \times 4c$
- Применяем связующие слои с **канальными масками**



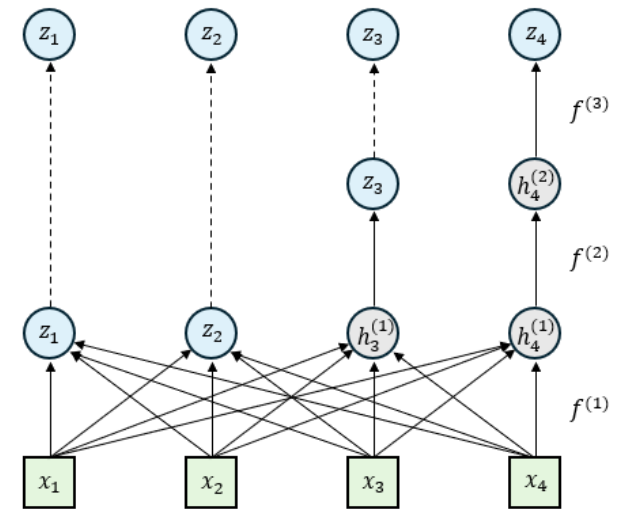
RealNVP

Проблема:

Прогонять весь входной вектор через все слои довольно затратно

Решение:

После каждого блока будем выносить половину переменных из потока сразу в финальный вектор $\mathbf{z}$



RealNVP: Samples



Glow

Glow: ActNorm

Glow – идейный наследник **RealNVP**, использующий **Affine Copling Layers** и **Multi – Scale** архитектуру

Авторы решили заменить эвристики **RealNVP** на более гибкие, обучаемые аналоги

- **BatchNorm** плохо работает с малыми размерами батчей (шумные статистики)
- Чтобы не зависеть от статистик батча, авторы предложили использовать **ActNorm** (**Activation Normalization**) слой

Glow: Invertible 1×1 Conv

Проблема:

Фиксированные перестановки в *RealNVP* не обучаются и не гарантируют оптимальное смешивание каналов

Решение:

Будем обучать обратимую свёртку 1×1 – линейный поток, который позволит модели самой найти оптимальный способ перемешивания каналов

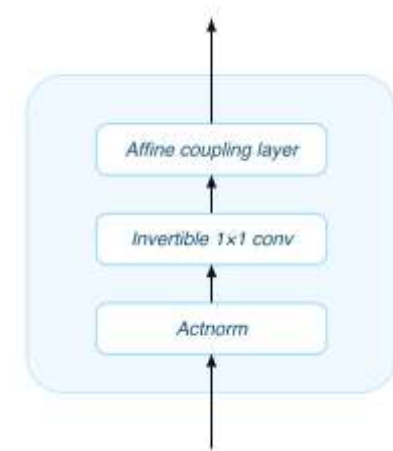
Трюк:

Чтобы быстро считать определитель матрицы свёртки W , используется *LU-разложение*

Glow: Flow Step

Glow объединяет все 3 операции в один обратимый блок (**Flow Step**), который многократно применяется в модели:

ActNorm $\rightarrow$ 1×1 Conv $\rightarrow$ Affine Coupling Layer



Multi – Scale архитектура:

- Сжимаем $h \times w \times c \rightarrow \frac{h}{2} \times \frac{w}{2} \times 4c$
- Делаем K шагов потока
- Половину каналов выносим сразу в финальный вектор $\mathbf{z}$

Glow: Samples



NICE → *RealNVP* → *Glow*

	<i>NICE</i> 2014	→	<i>RealNVP</i> 2016	→	<i>Glow</i> 2018
Слой связи	аддитивный $\mathbf{z}_2 = \mathbf{x}_2 + m_\theta(\mathbf{x}_1)$		аффинный $\mathbf{z}_2 = \mathbf{x}_2 \odot e^{s_\theta(\mathbf{x}_1)} + t_\theta(\mathbf{x}_1)$		аффинный
Меняет объём	только финальный слой масштаба		в каждом слое связи		в каждом слое связи
Нормализация	—		Batch Norm		ActNorm
Перемешивание	чередование разбиений		шахматная и канальная маски		обучаемая свёртка 1×1
Архитектура	композиция слоёв		Multi-scale		Multi-scale

 что изменилось по сравнению с предыдущей моделью

Спасибо за внимание!